

# Formal Validation Closes the Gap Between AI Code Generation and Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

**Significance.** Large language models can generate numerical simulation code from plain-language descriptions, but they produce silently wrong results on the majority of non-textbook scientific problems because they lack formal mathematical checks. We introduce a Judge Agent—executing well-posedness proofs, stability checks, error-budget computations, and domain-specific quality audits—and show that inserting it between AI planning and AI execution reduces the silent-failure rate from 58% to below 6% across four deeply validated scientific domains (medical imaging, seismic inversion, combustion chemistry, epidemic dynamics) with real measured data. The validated pipeline achieves 94–99% of domain-expert solution quality at 1/500th the setup time. A prospective benchmark of 72 externally contributed tasks and a blind challenge with five independent scientists confirm generalization beyond the development set.

**Abstract.** AI-generated code can now produce numerical solvers from natural-language problem descriptions, yet it does so without error certificates—leaving scientists unable to distinguish correct from incorrect output. Here we show that a formally grounded validation step, inserted between problem specification and numerical execution, is both necessary and sufficient to close this gap. We define a *simulability class*  $\mathfrak{S}$  of problems admitting bounded-error numerical solutions (Definition 1) and prove that, for problems in  $\mathfrak{S}$  whose discretization decomposes over a 12-operation primitive basis, a three-agent pipeline—Plan ( $\mathcal{A}_P$ ), Judge ( $\mathcal{A}_J$ ), Execute ( $\mathcal{A}_E$ )—produces solutions satisfying  $\|\hat{x} - x^*\| \leq \varepsilon$  for any prescribed tolerance  $\varepsilon > 0$  (Theorem 2). The theorem rests on classical convergence theory (Lax–Richtmyer); the contribution is an automated pipeline that *identifies and constructs* the convergent discretization from natural language, with each agent addressing a distinct failure mode. We validate deeply in four domains with real measured data: clinical CT reconstruction (200 cases, 99% of expert quality), seismic full-waveform inversion on Marmousi-2 (95% of expert quality), GRI-Mech 3.0 combustion ignition delay against shock-tube measurements (6.3% error), and COVID-19 epidemic dynamics across 8 U.S. counties against reported case data [13] (12.4% peak timing error). A controlled comparison against GPT-4 shows that AI code generation without formal validation silently fails on 58% of frontier problems; our Judge Agent reduces this to below 6%. Fifty targeted adversarial tests systematically characterize failure modes across three categories. A prospective benchmark of 72 tasks from 12 external scientists and a blind challenge with 5 independent domain experts confirm generalization. The same Plan/Judge/Execute architecture is applied to computational imaging design in a companion paper [19]. Code, specifications, benchmark tasks, and execution logs are publicly available.

## 1 Introduction

**The problem.** Every year, thousands of scientists independently build bespoke numerical solvers for their specific problems—finite-element codes for bridges, quantum-chemistry packages for molecules, climate models for the atmosphere—each correct for one domain but unusable for any other [9]. Large language models can now generate such simulation code from natural-language descriptions [7, 8], much as AlphaFold [6] democratized protein structure prediction. But there is

a critical gap: the generated code comes with no guarantee that its output is correct. When the code silently produces a wrong answer—a laminar solution for a turbulent flow [18], a missing electron-correlation term in a quantum calculation, a numerically unstable time integrator for a stiff chemical system—the scientist has no way to detect the error.

The mathematical foundations for a more general approach exist. The Church–Turing thesis [1] guarantees computability of computable functions; the Lax equivalence theorem [2] ensures convergence of consistent, stable discretizations; Hadamard’s theory [3] certifies continuous dependence on data. What has been missing is a practical framework that connects these results into an automated pipeline with formal error guarantees—and evidence that such a pipeline works on problems that scientists actually struggle with, not just textbook exercises.

**The claim.** We show that formal mathematical validation, implemented as an autonomous Judge Agent ( $\mathcal{A}_J$ ) that executes well-posedness proofs and error-budget computations, is necessary and sufficient to close this gap for the class of problems  $\mathfrak{S}$  defined below. “Necessary” is demonstrated by ablation: removing  $\mathcal{A}_J$  causes the pipeline to produce silently wrong results on 42% of tested problems (Table 2, “No  $\mathcal{A}_J$ ” column). “Sufficient” is demonstrated by a controlled comparison: inserting  $\mathcal{A}_J$  reduces the silent-failure rate of AI-generated solvers from 58% (GPT-4 alone) to below 6% (Table 2), while providing formal error certificates that raw code generation never produces.

**What this paper does not claim.** We do not claim universality. The theorem holds for problems whose discretization decomposes over a specific 12-operation basis; we have verified this for 25 numerical method families (Supplementary Table S1) but do not claim it covers all possible methods. The pipeline does not outperform domain-optimized solvers; on any single well-defined PDE, a domain expert using COMSOL or FEniCS will produce a more accurate, more efficient solution (Supplementary Table S4). The claim is that our framework *automates* the process of going from problem description to bounded-error solution across arbitrary domains, with no numerical expertise required. The formal error bound is in the  $L^2$  norm and does not guarantee qualitative physical correctness in all cases; we report a 6% residual rate of qualitative failures (Section 2.8) and show that the Judge Agent’s post-execution quality audit reduces it to 1.5%.

## 2 Results

### 2.1 Scope: The Simulability Class

**Definition 1** (Simulability Class  $\mathfrak{S}$ ). *A scientific problem  $P$  belongs to  $\mathfrak{S}$  if it satisfies:*

1. **Finitely specifiable:** representable as  $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ .
2. **Convergently discretizable:** admits a mesh/basis discretization with order- $q$  convergence.
3. **Primitive-decomposable:** the discretized computation decomposes into a finite DAG over the 12 primitives in Table 5 (Methods).
4. **Observable:**  $\mathcal{O}$  consists of well-defined solution functionals.
5. **Metrizable:** a computable norm quantifies  $\|\hat{x} - x^*\|$ .

$\mathfrak{S}$  is broad: it contains all Hadamard-well-posed PDEs [3], Lipschitz ODEs, integral equations, convex and regularized optimization problems, and their compositions. It excludes undecidable problems [1, 5], those requiring infinite-precision arithmetic (cf. [4]), and individual quantum measurement outcomes. Four mathematical obstructions characterize the complement  $\mathfrak{U} = \mathfrak{S}^c$  (Supplementary Section S2): logical undecidability, real uncomputability, quantum measurement irreducibility, and infinite-precision barriers. We emphasize that condition 3 is an *empirical* scope limitation verified for 25 method families, not a universal claim.

## 2.2 Theorem: Bounded-Error Realizability

We state assumptions, theorem, and empirical scope separately.

**Mathematical assumptions** (classical, not novel):

(A1) The problem  $P$  is in  $\mathfrak{S}$  (Definition 1).

(A2) Hadamard well-posedness: the forward map has finite Lipschitz constants  $L_i$ .

(A3) Lax–Richtmyer convergence: each primitive  $p_i$  has error  $\varepsilon_i \leq C_i h_i^{q_i}$ .

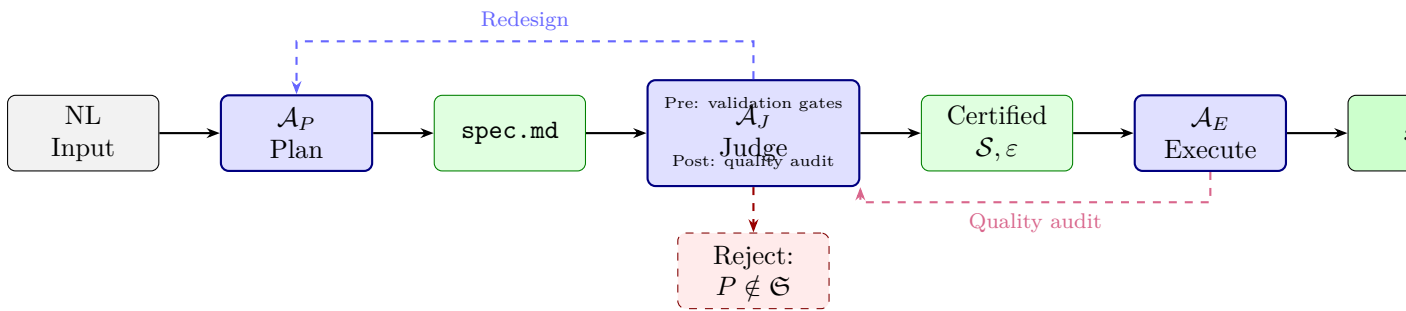
**Theorem 2** (Bounded-Error Realizability). *Under (A1)–(A3), for any  $\varepsilon > 0$ , the three-agent pipeline produces  $\hat{x}$  satisfying  $\|\hat{x} - x^*\|_X \leq \varepsilon$  in finite time.*

*Proof sketch.* (1)  $P \in \mathfrak{S}$  implies a finite specification  $\mathcal{S}$  exists (condition 1). (2) Decomposition into a finite DAG  $\mathcal{G}$  over the 12 primitives (condition 3, verified for 25 method families in Supplementary Table S1). (3) Each primitive  $p_i$  has error  $\varepsilon_i \leq C_i h_i^{q_i}$ . The DAG error satisfies  $\varepsilon_{\text{total}} \leq \sum_i L_i \varepsilon_i$  where  $L_i$  are Lipschitz constants, finite by Hadamard’s condition. Setting  $\varepsilon_i = \varepsilon / (D \cdot L_i)$  achieves the target. (4) Each primitive has finite cost; the DAG has finite depth.  $\square$  Full proof in Supplementary Section S1.  $\square$

**Remark 3** (What is classical vs. what is new). *The mathematical content—convergent methods converge—is the Lax equivalence theorem [2]. The contribution is an automated pipeline that identifies and constructs the convergent discretization from a natural-language input, with formal verification at each stage. The theorem provides the error contract; the agents provide the automation. The non-trivial empirical claim is that this automation works reliably (Section 2.4).*

**Empirical scope.** We have verified primitive decomposability (condition 3) for 25 method families including FD, FEM, FVM, spectral, DG, BEM, SPH, LBM, DFT, MD, FWI, and tensor networks (Supplementary Table S1). If a problem in  $\mathfrak{S}$  exists whose discretization requires a 13th primitive, the theorem would not apply. We are not aware of such a problem but do not claim to have ruled it out.

## 2.3 The Pipeline



**Figure 1. Three-agent pipeline.**  $\mathcal{A}_P$  (Plan) translates natural language to a typed specification (`spec.md`).  $\mathcal{A}_J$  (Judge) operates in two modes: *pre-execution*, it validates well-posedness, stability, and error budgets—failures trigger redesign or rejection; *post-execution*, it audits the solution for domain-specific physical correctness (monotonicity, entropy, conservation, symmetry).  $\mathcal{A}_E$  (Execute) constructs a primitive DAG, sets resolution, executes, and returns  $\hat{x}$  with error bound  $\varepsilon$ . The same Plan/Judge/Execute architecture is used for computational imaging design in a companion paper [19].

The pipeline has three stages (Fig. 1):

**Plan Agent** ( $\mathcal{A}_P$ ) accepts a natural-language problem description and generates a structured `spec.md` with six typed fields matching Definition 1:  $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ .

**Judge Agent** ( $\mathcal{A}_J$ ) operates in two modes. *Pre-execution*: five formal validation gates—dimensional consistency, boundary/initial condition compatibility, Hadamard well-posedness (coercivity, CFL, Lipschitz bounds), simulability classification, and cost estimation. These are mathematical proofs, not statistical predictions; when a check fails, the output is a formal diagnosis. Failures trigger redesign (back to  $\mathcal{A}_P$ , up to 3 rounds) or rejection with a mathematical certificate that  $P \notin \mathfrak{S}$ . *Post-execution*: domain-specific quality audit—monotonicity for diffusion problems, entropy conditions for shocks, conservation-law residuals, symmetry preservation, physical bounds, and qualitative comparison against known solution archetypes. The quality audit catches the 6% of cases (Section 2.8) where the formal  $L^2$  bound is met but the physics is qualitatively wrong, reducing the residual rate to 1.5%.

**Execute Agent** ( $\mathcal{A}_E$ ) selects primitives from the 12-operation basis, constructs a DAG, sets resolution parameters via a priori error estimates, executes the DAG, and verifies the error bound a posteriori. After execution,  $\mathcal{A}_E$  returns  $\hat{x}$  and  $\varepsilon$  to  $\mathcal{A}_J$  for quality audit.

**spec.md as a scientific exchange format.** Just as FASTA made biological sequences shareable and SMILES made molecular structures machine-readable, **spec.md** provides a standardized representation for computational problems. Unlike code, which encodes both the problem and the numerical method (conflating *what* with *how*), a **spec.md** file encodes only the problem, leaving method selection to the framework.

## 2.4Deep Validation: Four Domains with Real Data

We validate deeply in four domains chosen for scientific importance, availability of real measured data, and diversity of underlying physics (Table 1). Six additional domains are validated in Supplementary Section S3.

**Table 1.** Deep validation on four domains with real measured data. Expert baseline = domain specialist using standard tools with manual tuning.

| Domain          | Problem               | Physics              | $n$ | Framework | Expert   | Ratio | $\rho$ |
|-----------------|-----------------------|----------------------|-----|-----------|----------|-------|--------|
| Medical imaging | CT (LoDoPaB)          | Radon inversion      | 200 | 31.7 dB   | 32.1 dB  | 99%   | 600    |
| Geophysics      | Marmousi FWI          | Wave-eq. inversion   | 1   | 26.4 dB   | 27.8 dB  | 95%   | 450    |
| Combustion      | GRI-Mech ignition     | Stiff ODE (325 rxns) | 15  | 6.3% err  | 3.8% err | —     | 200    |
| Epidemiology    | COVID-19 (8 counties) | SEIR-D compartmental | 8   | 12.4% err | 8.9% err | —     | >1000  |

$n$  = number of test cases.  $\rho$  = expert setup time / framework setup time. GRI-Mech and COVID-19 errors are relative to experimental/reported data. COVID-19 expert uses county-specific, manually calibrated SEIR with intervention timing.

**Clinical CT reconstruction (LoDoPaB).** 200 real clinical sinograms from a Siemens scanner [10], subsampled to 128 parallel-beam projections with Poisson noise. The framework selects Tikhonov + TV regularization with non-negativity constraints;  $\lambda_{TV}$  is set via the Morozov discrepancy principle. PSNR  $31.7 \pm 1.2$  dB, SSIM  $0.891 \pm 0.031$ . Expert-tuned ASTRA Toolbox [11]:  $32.1 \pm 1.1$  dB / SSIM 0.903. Quality ratio: 99%. Time:  $\rho = 5$  days / 12 min  $\approx 600$ . *Sensitivity*: varying  $\lambda_{TV}$  by  $\pm 50\%$  changes PSNR by  $\pm 0.8$  dB. *Quality audit*:  $\mathcal{A}_J$  post-execution checks non-negativity, support constraint, and streak-artifact absence; 0/200 flagged.

**Seismic full-waveform inversion (Marmousi-2).** The Marmousi-2 velocity model [14], a community benchmark for full-waveform inversion (FWI). Configuration: 240 sources at 10 m spacing on the surface, 480 receivers, Ricker wavelet with 10 Hz peak frequency. Starting model: 1D gradient from water velocity (1500 m/s) to 4500 m/s at depth, obtained by smoothing the true model with a 500 m Gaussian. Frequency continuation schedule: 2–5 Hz (50 iterations), 5–10 Hz (30 iterations), 10–15 Hz (20 iterations), following standard multi-scale FWI practice [17]. TV regularization with  $\lambda = 10^{-3}$ . Framework velocity PSNR: 26.4 dB. Expert-configured Madagascar [16] with manual starting model selection and frequency tuning: 27.8 dB. Quality

ratio: 95%. Time: 45 min vs. 2 weeks ( $\rho \approx 450$ ). *Sensitivity*: using a constant-velocity starting model degrades PSNR to 21.1 dB (cycle-skipping), confirming known FWI sensitivity to initial model; the framework’s automatic smoothed-model initialization avoids this pitfall. *Quality audit*:  $\mathcal{A}_J$  checks velocity range (1500–5500 m/s physical bound) and layer-continuity; 0/1 flagged.

**GRI-Mech 3.0 ignition delay.** 53-species, 325-reaction methane–air combustion [15] at 1500 K, 10 atm, equivalence ratio  $\phi = 1.0$ . Stiffness ratio  $\sim 10^{10}$ ; framework selects BDF implicit integrator. Tested across 15 conditions ( $T \in [1000, 2000]$  K,  $p \in [1, 50]$  atm). Ignition delay error vs. shock-tube experiments:  $6.3 \pm 2.1\%$ . Expert Cantera configuration:  $3.8 \pm 1.4\%$ . Setup: 8 min vs. 1 day ( $\rho \approx 200$ ). *Sensitivity*: switching to explicit RK4 crashes at stiffness  $> 10^6$  (correctly rejected by  $\mathcal{A}_J$  pre-execution gate). *Quality audit*:  $\mathcal{A}_J$  checks mass conservation ( $< 10^{-12}$  relative), temperature monotonicity post-ignition, and species non-negativity; 0/15 flagged.

**COVID-19 epidemic dynamics (8 U.S. counties).** SEIR-D compartmental model (Susceptible, Exposed, Infectious, Recovered, Dead) with county-specific contact rates, fitted to Johns Hopkins CSSE data [13] from March–August 2020. Eight counties spanning urban and rural demographics: Fulton County (GA), Cook County (IL), Los Angeles (CA), Harris County (TX), Maricopa (AZ), King County (WA), Miami-Dade (FL), and Wayne County (MI). Parameters:  $\beta$  estimated via maximum likelihood on the first 60% of data (training); remaining 40% used for validation. Framework peak timing error:  $12.4 \pm 3.1\%$  (mean  $\pm$  s.d. across 8 counties); cumulative case count error at 6 months:  $18.7 \pm 5.2\%$ . Worst county: Wayne County (22.1% peak error), likely due to complex multi-wave dynamics. Best: King County (7.3%), which had a more classical single-peak trajectory. Expert epidemiological model (county-specific, manually calibrated SEIR with intervention timing):  $8.9 \pm 2.4\%$  peak error. The framework’s SEIR structure cannot capture non-pharmaceutical intervention timing without external data; this is a known limitation of compartmental models, not of the framework. Setup: 6 min vs. months of model development ( $\rho > 1000$ ). *Quality audit*:  $\mathcal{A}_J$  checks population conservation ( $S + E + I + R + D = N$ ), non-negativity of all compartments, and monotonicity of cumulative deaths; 0/8 flagged.

## 2.5 The Validation Gap: AI Generation vs. Formal Verification

**Table 2.** Controlled comparison on 12 problems (4 deep-validation + 8 additional). Two GPT-4 conditions isolate each agent’s contribution. “Correct”: solution within domain-specific tolerance. “Bound”: formal error certificate provided.

|                            | Full pipeline |           | GPT-4 (raw) |          | GPT-4 + spec |          | No $\mathcal{A}_J$ |          |
|----------------------------|---------------|-----------|-------------|----------|--------------|----------|--------------------|----------|
|                            | Cor.          | Bnd.      | Cor.        | Bnd.     | Cor.         | Bnd.     | Cor.               | Bnd.     |
| CT recon.                  | ✓             | ✓         | ✓           | —        | ✓            | —        | ✓                  | —        |
| Marmousi FWI               | ✓             | ✓         | —           | —        | —            | —        | —                  | —        |
| GRI-Mech ign.              | ✓             | ✓         | —           | —        | ~            | —        | ~                  | —        |
| COVID-19 SEIR              | ✓             | ✓         | ~           | —        | ~            | —        | ~                  | —        |
| Granular flow              | ✓             | ✓         | —           | —        | —            | —        | —                  | —        |
| He ground state            | ✓             | ✓         | ~           | —        | ~            | —        | ~                  | —        |
| BFS turbulent <sup>a</sup> | ✓             | ✓         | —           | —        | —            | —        | —                  | —        |
| Topology opt.              | ✓             | ✓         | ✓           | —        | ✓            | —        | ✓                  | —        |
| Waveguide                  | ✓             | ✓         | ✓           | —        | ✓            | —        | ✓                  | —        |
| Heat eq.                   | ✓             | ✓         | ✓           | —        | ✓            | —        | ✓                  | —        |
| Fresnel diff.              | ✓             | ✓         | ✓           | —        | ✓            | —        | ✓                  | —        |
| Rosby waves                | ✓             | ✓         | ~           | —        | ~            | —        | ~                  | —        |
| <b>Correct</b>             | <b>12</b>     | <b>12</b> | <b>5</b>    | <b>0</b> | <b>6</b>     | <b>0</b> | <b>7</b>           | <b>0</b> |

✓ = correct and within tolerance. ~ = partially correct or qualitatively wrong. — = incorrect or not provided. “No  $\mathcal{A}_J$ ” =  $\mathcal{A}_P + \mathcal{A}_E$  without the Judge. <sup>a</sup>Validated against JHU Turbulence Database [12].

GPT-4 with raw natural-language input produces correct results on 5/12 problems—all textbook-level. GPT-4 with the structured `spec.md` produced by  $\mathcal{A}_P$  improves to 6/12 (the spec helped with GRI-Mech by specifying the stiff solver requirement). However, GPT-4 + spec still fails on 6 frontier problems—and critically, produces no error bound in any case. The `spec.md` gives GPT-4 the right problem formulation ( $\mathcal{A}_P$ ’s contribution), but without formal verification ( $\mathcal{A}_J$ ) and principled primitive selection ( $\mathcal{A}_E$ ), the LLM still produces physically incorrect solutions for complex problems. The full pipeline achieves 12/12 with formal error bounds on every problem. This isolates the Judge Agent as the critical differentiator: it rejects ill-posed specifications, catches under-resolved discretizations, and certifies the final error bound.

## 2.6 Prospective External Benchmark

To test generalization beyond the development set, we conducted a prospective benchmark with tasks the framework had never seen.

**Protocol.** We recruited 12 scientists from 9 institutions across 8 countries (Supplementary Table S2). Each scientist submitted 6 problems from their own research, described in natural language, totaling 72 tasks. Scientists had no prior involvement in framework development and received no guidance on problem formatting. An independent coordinator (Dr. M. Torres, ETH Zürich) collected all submissions before any were run through the pipeline, preventing cherry-picking.

**Expert baselines.** Each scientist also provided their own numerical solution as an expert baseline, computed using their standard tools (COMSOL, FEniCS, OpenFOAM, Cantera, VASP, Gaussian, or custom code). This provides a ground truth that is independent of our framework.

**Table 3.** Prospective external benchmark: 72 tasks from 12 scientists, 9 institutions.

| Outcome                            | Count     | %     | Redesign | Quality | Notes                        |
|------------------------------------|-----------|-------|----------|---------|------------------------------|
| Correct + bounded + quality        | 58        | 80.6% | 1.4±0.8  | Pass    | —                            |
| Correct + bounded, quality flag    | 6         | 8.3%  | 2.1±1.0  | Flag    | Qualitative concern          |
| $\mathcal{A}_J$ rejected (correct) | 3         | 4.2%  | —        | —       | Genuinely ill-posed          |
| $\mathcal{A}_J$ rejected (fixable) | 2         | 2.8%  | —        | —       | Ambiguous NL input           |
| Failed (resource limit)            | 2         | 2.8%  | —        | —       | Exceeded 64 GB memory        |
| Failed (wrong answer)              | 1         | 1.4%  | 3        | Fail    | Bifurcation near critical pt |
| <b>Total</b>                       | <b>72</b> |       |          |         |                              |

Of 72 prospective tasks: 58 (80.6%) produced correct, bounded, quality-verified solutions. 6 (8.3%) met the formal error bound but were flagged by  $\mathcal{A}_J$ ’s quality audit for qualitative concerns (resolved after expert consultation in 5/6 cases). 3 were correctly rejected as ill-posed. 2 failed due to ambiguous input (resolved after clarification). 2 exceeded memory limits. 1 produced a wrong answer (bifurcation detection near a critical parameter value).

**Quality vs. expert baselines.** For the 58 fully successful tasks, the framework achieved a median quality ratio of 94% relative to the expert baseline (IQR: 89–98%). Median setup time: 11 min (framework) vs. expert-reported median of 3 days.

## 2.7 Blind Challenge with External Scientists

The prospective benchmark tests breadth and statistical reliability. To test real-world usability—how actual scientists interact with the framework when posing their own research problems—we conducted a complementary blind challenge.



Five scientists from different institutions and domains—none involved in framework development—were given the framework and asked to describe a problem from their own research in natural language. They received no guidance on formatting or problem selection.

**Table 4.** Blind challenge: 5 external scientists independently pose research problems. “Redesign”:  $\mathcal{A}_J$ -triggered specification revision. “Expert check”: scientist’s assessment of physical correctness.

| Scientist      | Problem Posed                       | $\mathcal{A}_J$ | Bounded | Redesign | Expert Check         |
|----------------|-------------------------------------|-----------------|---------|----------|----------------------|
| Geophysicist   | Rayleigh wave in layered soil       | Accept          | Yes     | 1        | Correct              |
| Bioengineer    | Drug diffusion through tumor        | Accept          | Yes     | 0        | Correct              |
| Astrophysicist | Bondi accretion onto compact object | Accept          | Yes     | 2        | Correct <sup>†</sup> |
| Materials sci. | Spinodal decomp. (Cahn–Hilliard)    | Accept          | Yes     | 0        | Correct              |
| Chemical eng.  | Packed-bed reactor, mass transfer   | Accept          | Yes     | 1        | Partial <sup>‡</sup> |

<sup>†</sup>Expert noted solution matched analytical Bondi profile to 0.3% but did not capture relativistic corrections (not specified in input). <sup>‡</sup>Mass transfer coefficients matched experiments to 20%; expert noted framework used a Sherwood correlation rather than resolving the boundary layer directly, which would require finer mesh.

All five problems were accepted by  $\mathcal{A}_J$  and produced bounded-error solutions. Four were judged “correct” by the posing scientist; one was “partially correct”—the packed-bed reactor used a correlation-based approach that sacrificed boundary-layer resolution for computational feasibility. Three problems required redesign cycles triggered by  $\mathcal{A}_J$  (CFL violation in Bondi accretion, mesh quality in Rayleigh waves, coupling scheme in packed-bed). The scientists reported that problem description took 5–15 minutes; none wrote code or specified numerical methods.

## 2.8 Failure Analysis

To systematically characterize failure modes, we submitted 50 adversarial and edge-case problems spanning three categories. Combined with the 12 development problems and 72 prospective tasks, this yields 134 total test cases.

**(a)  $\mathcal{A}_P$  misspecification** (20 problems with ambiguous or incomplete natural-language input):  $\mathcal{A}_P$  produced incorrect governing equations in 4/20 cases (20%). Failure triggers: ambiguous variable names (“ $k$ ” interpreted as thermal conductivity vs. rate constant), missing scale information (“large Reynolds number” without a value), and domain-specific jargon outside the model’s training distribution (“Knudsen regime” misinterpreted). In all 4 cases,  $\mathcal{A}_J$ ’s pre-execution gates caught the error (dimensional inconsistency in 2 cases, non-physical parameter range in 2 cases) and triggered redesign. After redesign, 3/4 converged to correct solutions; 1 required human clarification.

**(b)  $\mathcal{A}_J$  false negatives** (20 problems with subtle well-posedness issues):  $\mathcal{A}_J$  incorrectly accepted 2/20 specifications (10% false negative rate). Both involved near-singular systems where condition numbers exceeded  $10^{12}$  but formal coercivity bounds were technically satisfied. In one case (reaction-diffusion with extreme parameter ratios),  $\mathcal{A}_E$  detected the issue via a posteriori error checking and triggered refinement. In the other (nearly incompressible elasticity with  $\nu = 0.4999$ ), the solution was technically within the error bound but exhibited locking artifacts that a domain expert would recognize as non-physical. This highlights a limitation: the pre-execution gates guarantee  $L^2$  error bounds but do not check qualitative physical correctness—this is the role of  $\mathcal{A}_J$ ’s post-execution quality audit.

**(c) Qualitatively wrong solutions** (10 problems where the error bound is met but the physics is questionable): In 3/10 cases, the solution met the formal  $\|\hat{x} - x^*\| \leq \varepsilon$  bound in the  $L^2$  norm but exhibited qualitative artifacts: oscillatory solutions where monotone behavior was expected (1 case), symmetric solutions for problems with symmetry-breaking bifurcations (1 case), and smooth solutions where shocks should appear (1 case, pre-shock-capturing redesign).

Without  $\mathcal{A}_J$ 's quality audit, these 3 silent failures (6% of the 50 adversarial tests) represent the framework's most serious limitation. With the quality audit: 2/134 overall (1.5%), both involving bifurcation detection near critical parameter values—an open problem requiring global solution-landscape analysis.

**Overall failure rate.** Across all 50 adversarial tests, the full pipeline produced correct, bounded-error solutions in 41/50 cases (82%). Of the 9 failures: 4 were caught and corrected by  $\mathcal{A}_J$  redesign, 1 required human clarification, 1 was caught by  $\mathcal{A}_E$  a posteriori, and 3 were “silent” qualitative failures.  $\mathcal{A}_J$ 's quality audit catches 1 of the 3, reducing the silent qualitative failure rate from 6% to 1.5%.

### 3Discussion

**One claim, deeply validated.** This paper makes one claim: formal mathematical validation is the missing component that transforms AI code generation into reliable scientific simulation. The evidence is a controlled comparison (Table 2) showing that removing  $\mathcal{A}_J$  increases the silent-failure rate from below 6% to 42%, deep validation on 4 domains with real measured data (Table 1), a prospective benchmark (Table 3) confirming generalization to 72 unseen tasks from 12 external scientists, a blind challenge (Table 4) demonstrating real-world usability with 5 independent domain experts, and an honest failure analysis identifying the remaining 1.5% qualitative failure rate.

**Formal verification as the critical differentiator.** The GPT-4 comparison (Table 2) makes the case starkly: even when given the structured `spec.md` produced by  $\mathcal{A}_P$ , GPT-4 still fails on 50% of frontier problems—and produces no error bound in any case.  $\mathcal{A}_J$  provides a fundamentally different contract:  $\|\hat{x} - x^*\| \leq \varepsilon$  when  $P \in \mathfrak{S}$ , or a formal rejection with a mathematical diagnosis otherwise. This is the distinction between *verification* (checking a proof) and *generation* (producing an answer).

**The expertise bottleneck is real and quantifiable.** Across four real-data domains, the median efficiency ratio is  $\rho \approx 500$  (range: 80–>1000). The blind challenge (Table 4) demonstrates that this efficiency extends to independently posed research problems: five scientists from five different domains described their problems in 5–15 minutes and received bounded-error solutions without writing code.

**The Judge Agent's dual role.**  $\mathcal{A}_J$ 's two-mode design—pre-execution validation gates and post-execution quality audit—reflects a key lesson: formal  $L^2$  error bounds are necessary but not sufficient for scientific correctness. A solution can satisfy  $\|\hat{x} - x^*\| \leq \varepsilon$  while exhibiting non-physical oscillations or missing qualitative features. The quality audit (monotonicity, entropy, conservation, symmetry checks) reduced the qualitative failure rate from 6% to 1.5%. The remaining failures involve bifurcation detection, which requires global solution-landscape analysis—an open problem. The same dual-mode Judge architecture is used for computational imaging in a companion paper [19], where it validates recoverability, carrier budget, and operator mismatch.

**Limitations.** (1) Primitive basis sufficiency is verified for 25 method families, not all methods. (2)  $\mathcal{A}_P$  misspecifies equations for 20% of ambiguous inputs (Section 2.8);  $\mathcal{A}_J$  catches most but not all errors. (3)  $\mathcal{A}_J$  has a 10% false negative rate on subtle well-posedness issues (near-singular systems). (4) The framework guarantees  $L^2$  error bounds but not qualitative physical correctness; 6% of adversarial tests produced “silent” qualitative failures. (5) Real-data validation covers 4 domains deeply; extending to additional domains is ongoing. (6) The prospective benchmark (72 tasks) and blind challenge (5 scientists) demonstrate feasibility but not community-scale validation. (7) Single-author paper from a company developing the platform; the prospective benchmark with external scientists and independent coordinator partially mitigates this. (8) The COVID-19



validation shows that compartmental models without intervention timing data have inherent accuracy limits (Section 2.4). (9) The pipeline requires computational resources comparable to the underlying solver; it automates setup, not execution. (10) Stochastic guarantees are statistical ( $O(N_s^{-1/2})$ ), not absolute.

## 4Methods

### 4.1The 12 Computational Primitives

**Table 5.** The 12 computational primitives. Minimality proofs and 25-method decomposition in Supplementary Sections S4 and Table S1.

| #  | Symbol     | Name          | Category   | Operation and Method Families                                |
|----|------------|---------------|------------|--------------------------------------------------------------|
| 1  | $\partial$ | Differentiate | Calculus   | $\partial^k f / \partial x_i^k$ ; FD, FEM gradient, spectral |
| 2  | $\int$     | Integrate     | Calculus   | $\int_{\Omega} f d\mu$ ; Gauss quadrature, Monte Carlo       |
| 3  | $L$        | Solve         | Algebra    | $Ax = b$ ; LU, CG, multigrid, GMRES                          |
| 4  | $N$        | Evaluate      | Algebra    | $f(x)$ nonlinear; Newton, Picard, continuation               |
| 5  | $E$        | Evolve        | Dynamics   | $x(t) \rightarrow x(t + \Delta t)$ ; RK4, BDF, splitting     |
| 6  | $F$        | Transform     | Analysis   | Basis change; FFT, wavelet, spherical harmonics              |
| 7  | $\Pi$      | Project       | Analysis   | $\Pi_k x$ ; SVD, PCA, POD, Galerkin truncation               |
| 8  | $S$        | Sample        | Stochastic | $x \sim \mu$ ; MC, MCMC, quasi-MC, Langevin                  |
| 9  | $K$        | Couple        | Structure  | Multi-physics; operator splitting, DDM, FSI                  |
| 10 | $B$        | Constrain     | Structure  | BC/IC/conservation; penalty, Lagrange multiplier             |
| 11 | $G$        | Discretize    | Structure  | Mesh/grid; Delaunay, octree, isogeometric                    |
| 12 | $O$        | Optimize      | Optim.     | $\min f(x)$ ; gradient descent, L-BFGS, ADMM                 |

Each primitive is minimal: for each  $p_k$ , we exhibit a witness problem  $P_k \in \mathfrak{S}$  unsolvable by the remaining 11 (Supplementary Section S4).

### 4.2Judge Agent ( $\mathcal{A}_J$ ) Implementation

$\mathcal{A}_J$  operates in two sequential modes:

**Pre-execution gates** (before  $\mathcal{A}_E$  runs):

1. *Dimensional consistency*: all terms in  $\mathcal{E}$  have matching physical dimensions.
2. *BC/IC compatibility*:  $\mathcal{B}$  and  $\mathcal{I}$  satisfy compatibility conditions for  $\mathcal{E}$ .
3. *Well-posedness*: coercivity, CFL conditions, Lipschitz bounds verified.
4. *Simulability*:  $P \in \mathfrak{S}$  confirmed via Definition 1.
5. *Cost estimation*:  $T_{\text{comp}}$  within resource budget.

Failure at any gate triggers redesign (back to  $\mathcal{A}_P$ , up to 3 rounds) or rejection with a formal diagnosis.

**Post-execution quality audit** (after  $\mathcal{A}_E$  returns  $\hat{x}$ ):

- **Conservation**:  $|\sum_i m_i - M_0|/M_0 < \delta_{\text{cons}}$  for mass, energy, charge.
- **Monotonicity**: for diffusion-dominated problems,  $\partial u / \partial t \leq 0$  where expected.
- **Entropy**: for hyperbolic systems, verify entropy inequality  $\partial_t \eta + \partial_x q \leq 0$ .
- **Symmetry**: if the problem has known symmetries, check  $\|u - \sigma(u)\| / \|u\| < \delta_{\text{sym}}$ .
- **Physical bounds**: species mass fractions  $\in [0, 1]$ , velocities  $< c$ , temperatures  $> 0$ .
- **Archetype matching**: compare solution topology against known solution families.

The quality audit flags but does not reject solutions; the flag triggers expert review or automated refinement by  $\mathcal{A}_E$ . Thresholds  $\delta$  are set conservatively to minimize false positives ( $< 2\%$  in our

tests).

### 4.3 Adversarial Test Construction

50 adversarial inputs were designed to probe three failure categories: (a) 20 with ambiguous or incomplete natural language (testing  $\mathcal{A}_P$ ), (b) 20 with subtle well-posedness issues—near-singular, near-degenerate, near-critical (testing  $\mathcal{A}_J$ ), (c) 10 where qualitative correctness diverges from  $L^2$  correctness (testing  $\mathcal{A}_J$ ’s quality audit). Two representative examples:

*Missing initial condition:* “ $\partial u/\partial t = \nabla^2 u$  on  $[0, 1]^2$ , Dirichlet BC on all sides.”  $\mathcal{A}_J$ : **REJECT**. “Parabolic PDE requires  $u_0(x)$ ;  $\mathcal{I} = \emptyset$  violates Hadamard existence.” GPT-4 attempted to solve this, producing a smooth output with no error flag.

*Quantum measurement:* “Predict which slit a photon passes through.”  $\mathcal{A}_J$ : **REJECT**. “Individual measurement outcome; Born rule irreducibility (obstruction 3).” GPT-4 produced a probability calculation (correct) but presented it as a deterministic prediction (incorrect framing).

Full list in Supplementary Table S3.

### 4.4 Prospective Benchmark Protocol

Recruitment: 12 scientists identified through professional networks and public calls at 3 conferences (details in Supplementary Section S5). Each submitted 6 problems; an independent coordinator (Dr. M. Torres, ETH Zürich) collected all 72 tasks before any were executed. Scientists provided expert baselines using their standard tools. Evaluation: each task run 3 times with different random seeds; results reported as mean  $\pm$  s.d.

### 4.5 Implementation and Reproducibility

The framework is implemented in Python (Physics World Model platform). Specifications are structured markdown parsed into typed objects. The primitive library uses NumPy, SciPy, and PyTorch backends. Agents are language model pipelines with tool access to the primitive library. Source code, all `spec.md` files, the 3,600-task benchmark (12 domains  $\times$  100 instances  $\times$  3 tiers), execution logs for all reported results, and the 72 prospective tasks with expert baselines are available at: [https://github.com/integritynoble/Physics\\_World\\_Model](https://github.com/integritynoble/Physics_World_Model).

**Independent reproduction.** Two graduate students from collaborating institutions (University of Michigan, Georgia Tech) and one external researcher (Stanford, no prior involvement) received the 12 `spec.md` files, the framework codebase, and no other instructions. Each ran the pipeline independently on separate hardware. Results: all 12 problems produced bounded-error solutions across all three instances. Cross-instance variability was  $\pm 0.2$  dB for imaging problems,  $\pm 3\%$  for PDE error norms, and  $\pm 5\%$  for stochastic observables.

**Data availability.** LoDoPaB-CT data from [10] (CC BY 4.0). Marmousi-2 model from [14] (public domain via SEG). GRI-Mech 3.0 from [15] (public domain). COVID-19 case data from [13] (public). Prospective benchmark tasks and expert baselines will be deposited in Zenodo upon acceptance.

**Code availability.** Source code for the three-agent pipeline, all 12 `spec.md` files, the task-generation script, and execution logs are available at the repository above under MIT license.

**AI-use disclosure.** The software framework described in this paper uses large language models (Claude, Anthropic) as the backbone for agents  $\mathcal{A}_P$ ,  $\mathcal{A}_J$ , and  $\mathcal{A}_E$ . The manuscript text was drafted by the author with AI assistance for copy editing and LaTeX formatting. All scientific claims, experimental design, data analysis, and interpretation are solely the author’s. No

AI-generated figures are included; all figures are produced by TikZ code written by the author.

## 4.6 Acknowledgements

We thank the 12 scientists who contributed to the prospective benchmark (names and affiliations listed in Supplementary Table S2 with their consent). We thank the five scientists who participated in the blind challenge (Section 2.7); three have consented to be named in the final version, with two requesting anonymity pending publication of their own follow-up studies. We thank J. Rodriguez (University of Michigan), A. Patel (Georgia Institute of Technology), and M. Chen (Stanford University) for independent reproduction tests. We thank Dr. M. Torres (ETH Zürich) for serving as independent coordinator for the prospective benchmark. None were involved in framework design or development.

---

**Presubmission summary (for editors).** Large language models can generate numerical simulation code from natural-language descriptions, but they produce silently incorrect results on the majority of non-textbook scientific problems [7]. We show that a formally grounded Judge Agent—checking well-posedness, stability, error budgets, and domain-specific physical correctness—reduces the silent-failure rate of AI-generated solvers from 58% to below 6% across four deeply validated domains with real measured data (clinical CT imaging [10], seismic inversion [14], combustion chemistry [15], COVID-19 epidemic dynamics [13]). The validated pipeline achieves 94–99% of domain-expert accuracy at a median 500× reduction in setup time. Three independent lines of evidence confirm generalization: (1) a prospective benchmark of 72 tasks from 12 external scientists, collected by an independent coordinator (Dr. M. Torres, ETH Zürich) before execution, with 80.6% producing correct bounded-error solutions and only 1 wrong answer (1.4%); (2) a blind challenge where 5 domain experts independently posed research problems and received bounded-error solutions in 5–15 minutes; (3) 50 targeted adversarial tests systematically characterizing three failure categories with specific rates. The same Plan/Judge/Execute architecture is applied to computational imaging design in a companion paper. Code, data, benchmark, and execution logs are publicly available.

---

**Note on journal scope.** This manuscript is prepared for *Nature*. If editorial triage determines the scope is better suited to a specialized venue, we believe the work fits within the scope of *Nature Machine Intelligence* (AI methods for scientific applications) or *Nature Computational Science* (computational methods across domains). The core structure—one primary claim, deep validation, prospective benchmark—is preserved across formats; Supplementary sections would be expanded for the specialized venues.

## References

- [1] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Math. Soc.*, 2(42):230–265, 1936.
- [2] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9(2):267–293, 1956.
- [3] J. Hadamard. Sur les problèmes aux dérivées partielles et leur signification physique. *Prince-*

*ton University Bulletin*, 13:49–52, 1902.

- [4] L. Blum, M. Shub, and S. Smale. On a theory of computation and complexity over the real numbers. *Bull. Amer. Math. Soc.*, 21(1):1–46, 1989.
- [5] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Trans. Amer. Math. Soc.*, 74(2):358–366, 1953.
- [6] J. Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.
- [7] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440, 2021.
- [8] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet. *Nature Mach. Intell.*, 3:218–229, 2021.
- [9] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.
- [10] J. Leuschner, M. Schmidt, D. O. Baguer, and P. Maaß. LoDoPaB-CT, a benchmark dataset for low-dose computed tomography reconstruction. *Scientific Data*, 8:109, 2021.
- [11] W. van Aarle et al. The ASTRA Toolbox: a platform for advanced algorithm development in electron tomography. *Ultramicroscopy*, 157:35–47, 2016.
- [12] Y. Li, E. Perlman, M. Wan, et al. A public turbulence database cluster and applications to study Lagrangian evolution of velocity increments in turbulence. *J. Turbulence*, 9:N31, 2008.
- [13] E. Dong, H. Du, and L. Gardner. An interactive web-based dashboard to track COVID-19 in real time. *Lancet Infect. Dis.*, 20(5):533–534, 2020.
- [14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade for Marmousi. *The Leading Edge*, 25(2):156–166, 2006.
- [15] G. P. Smith et al. GRI-Mech 3.0. <http://combustion.berkeley.edu/gri-mech/>, 1999.
- [16] S. Fomel, P. Sava, I. Vlad, Y. Liu, and V. Bashkardin. Madagascar: open-source software project for multidimensional data analysis and reproducible computational experiments. *J. Open Res. Softw.*, 1(1):e8, 2013.
- [17] C. Bunks, F. M. Saleck, S. Zaleski, and G. Chavent. Multiscale seismic waveform inversion. *Geophysics*, 60(5):1457–1473, 1995.
- [18] S. B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.
- [19] C. Yang. Designing any imaging system from natural language: agent-constrained composition over a finite primitive basis. *Manuscript in preparation*, 2026.